Architectural Guardrails: An Opinionated Framework for Preventing Structural Decay in Greenfield Research Software

Art O'Liathain
22363092@studentmail.ul.ie
University of Limerick

May 2026

Keywords Static Analysis · Machine Learning · Architectural Technical Debt · Maintability · Research Software · Temporal Coupling · Random Forests · Hueristic Based Dataset Generation

Contents

1 Abstract . . . 3
2 Introduction . . . 3
3 Background Research . . . 4
3.1 The current state of research software . . . 4
3.2 Maintainability . . . 5
3.2.1 What does it mean to be maintainable? . . . 5
3.2.2 Measuring Maintainability . . . 6
3.2.3 Code churn's impact on maintainability . . . 8
3.3 Static Analysis . . . 8
3.3.1 Current Methods . . . 9
3.4 Machine learning . . . 9
3.4.1 Decision Trees . . . 10
3.4.2 Random Forest . . . 10
3.4.3 Applications to maintainability detection . . . 11
3.5 Summary . . . 11
4 Methodology . . . 13
4.1 Dataset Selection and Acquisition . . . 13
4.1.1 Data collection . . . 13
4.1.2 Dataset First Pass Analysis . . . 15
4.2 Dataset Labeling . . . 16
4.2.1 Hub Score . . . 17
4.2.2 Contrastive sampling . . . 17
4.2.3 Addressing Data Leakage . . . 18

4.2.4 Dataset Summary . . . 18
4.3 Feature Engineering . . . 18
4.4 Model Implementation and Training . . . 20
4.5 Rule Derivation Process . . . 20
4.6 Evaluation Framework . . . 20
5 Future Work . . . 21
Bibliography . . . 21

List of Figures

Figure 1 Pre filter Distribution of Collaborators . . . 14
Figure 2 Pre filter Distribution of Commits . . . 15

List of Tables

Table 1 Extracted code metrics . . . 19

1 Abstract

2 Introduction

Research software is almost always seen as a means to an end, a tool used to prove a hypothesis so a paper can be published. When you combine this mindset with the fact that roughly 90% of developers are rely on learning code through self teaching (Hannay *et al.*, 2009), it leads to an epidemic of sub-optimal code within research projects.

This has cascading effects: while the code might work just well enough for one paper, it lacks longevity. This means future researchers often have to reimplement previous work just to start the next step. This is a difficult task, considering that on average only 25% of code repositories can run without issues or modifications (Trisovic *et al.*, 2022). It also raises an insidious question: if researchers produce results using a code artifact that cannot be reproduced, they may be producing results that cannot be verified.

This is why tools like static analysis are necessary to raise the floor of code quality. Unfortunately, these warnings are often ignored due to high volume (Johnson *et al.*, 2013). Warnings are not enough; there needs to be an opinionated system that uses empirical data to accurately predict if a file will become a maintenance issue before it is added to a larger codebase. Expert-driven rules attempt to generalize across all domains, but this breadth forces them to be relegated to low-priority warnings to accommodate architectural variance; an empirical, data-driven approach avoids this trap.

This paper seeks to answer the question: Can opinionated, objective rules be derived from historical and static analysis on code repositories using machine learning?

To approach this, an explainable machine learning approach using decision trees and random forests will be used. To combat the limitation of requiring a manually labeled dataset, an automatic approach using heuristic-based methods will be used to quickly curate a high-quantity dataset from research files sampling from over 200+ different research repositories resulting in a high quality dataset of over 4,000 files. This allows the ML models to learn patterns specific to research software, increasing the confidence in the resulting quality gate.

3 Background Research

3.1 The current state of research software

Research software is software made to investigate new ideas and concepts. This comes with one very prominent issue, researchers who are leading the way to new concepts are generally not programmers by trade, 90% of researchers and graduates rely on self teaching in programming (Hannay *et al.*, 2009) Software is made without maintainability in mind only following the check of "It works on my computer". This is best observed by (Trisovic *et al.*, 2022) where over 9000 R files were analysed from the Dataverse Project (http://dataverse.org/). In this study only 25% of R projects were runnable initially, after performing code cleaning, inferring dependencies and altering hardcoded paths 56% were able to be run. This is a shocking finding, as this is irrespective of the code output, only having a successfully run of the code was incredibly inconsistent. (Trisovic *et al.*, 2022) did note that journals are beginning to mandate better quality standards for code through research software standards to particularly allow for reproducibility to ensure the results published by the papers are correct.

Researchers and graduates need clear guidance on how to create and maintain a large software project due to their lack of formal training (Hannay *et al.*, 2009). Unfortunately, mainstream research software guidance remains overwhelmingly focused on reproducibility (Wilson, 2006), (Wilson *et al.*, 2017; Jiménez *et al.*, 2017; Marwick, 2017), often relegating other critical facets (such as maintainability and performance) to secondary considerations. While industry utilizes mature tooling and standards to balance reproducibility with code health, academic guidance often fails to provide actionable criteria for software architecture or long-term maintenance. This lack of structural guidance directly contradicts the sustainability goals for research software proposed by (Howison and Herbsleb, 2014).

To compound the problem more research software is made to fit a funding cycle without a care for the longevity of the codebase. Funding is fundamentally not suited for research software (Howison and Herbsleb, 2014) notes that unlike commercial products where revenue grows linearly with users, research grants are fixed and do not require or reward researchers for user acquisition. Funding pushes research software to focus on itself with no incentives outside of a research paper output. A flawed approach which (Howison and Herbsleb, 2014) does address with two alternative solutions; Commercial models in which the research software itself transitions to a self sufficient project which keeps itself alive and peer-production models which is a similar approach as open-source. Both of these models have merit, they allow research software to live past the funding cycle and continue to improve over time. However, there is one key issue: both rely on the existence of a maintainable codebase. If a project is built with significant 'technical debt' during the funding cycle, the cost of maintenance becomes an insurmountable barrier. A commercial spin-off will fail under the weight of high development costs, and a peer-production community will fail to form because outside contributors cannot navigate or

extend a poorly structured codebase. Thus, without a foundation of coding standards, these sustainability models remain aspirational rather than achievable.

Addressing the core issue of research software longevity while adhering to the constraints governing research software as a whole is the gap this paper is seeking to fill. To create an opinionated static analysis tool that will ensure code written has a floor of quality it cannot dip below through objective quality gates. By automating the "quality check" process, the quality of research software maintainability can be raised without requiring researchers to become professional developers. Static analysis provides the immediate, clear guidance that current literature lacks, ensuring that software is not just reproducible today, but maintainable and extensible for the researchers of tomorrow.

3.2 Maintainability

To raise code quality, maintainability must be defined. Maintainability is a term used frequently in software engineering, there is agreed upon definition on what maintainability is but ISO25010 defines it as "The degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements." It defines that the sub-sections of maintainability are *Modularity, Re-usability, Analysability, Modifiability, Testability.* (*ISO 25010*, no date) This is a very broad definition which essentially defines requirements for how easy can the system be modified for change. Due to the broadness of the definition it can easily be used to bring other non functional requirements as sub requirements ie. Flexibility, Adaptability, Scalability, Fault tolerance, Learn ability.

The lack of clear testable outcomes for each quality in the ISO standard leads to a conceptual overlap where maintainability becomes a 'catch-all' category for non-functional requirements, making assessing what is truly maintainable very difficult.

3.2.1 What does it mean to be maintainable?

To label code as maintainable, modularity is one key factor in the assessment. Modularity done correctly facilitates changeability through logical decomposition of functionality, it may seem simple yet if done incorrectly can have long lasting consequences on the maintainability and longevity of a codebase. D.L Parnas presented a study revealing how easily developers could fall into the pitfall of designing the system in a human like manner (Parnas, 1972). Designing systems based on a humans natural flow through a system, converting those actions into a flowchart of logical operations is a method that creates code that is not resilient. While it may be modular on paper as every "step" would be logically separate the code can still be highly coupled through cross cutting dependencies which do not conform to the human flow, impacting the maintainability of the codebase. An example of this would be

The effect has been substantiated by (Cai *et al.*, 2025) with a direct correlation being made between coupling and maintainability overhead. The core purpose of modularisation shouldn't be encapsulating subroutines into modules. Design decisions that are likely

to change should serve as the foundational points to creating modular code, allowing subroutines to be compositions of modules. Taking modularity into account will serve as a solid baseline to assess the maintainability of a codebase and files.

3.2.2 Measuring Maintainability

As maintainability is an abstract concept, various frameworks have attempted to reduce it to a concrete numerical metric. An early approach in this area is the Maintainability Index (MI), which calculates a score based on a weighted combination of Halstead Volume, cyclomatic complexity, lines of code (LOC), and comment density (Oman and Hagemeister, 1992). The formula typically utilized for this assessment is:

$$\mathrm{MI} = 171 - 5.2\ln(V) - 0.23(G) - 16.2\ln(\mathrm{LOC}) + 50\sin\left(\sqrt{2.46 \times C}\right) \tag{1}$$

Where:

- V represents **Halstead Volume**;
- G is **Cyclomatic Complexity**;
- C is the **percentage of comment lines**.

(Oman and Hagemeister, 1992) validated this metric through feedback from 16 developers and empirical testing on industrial systems, including those at Hewlett-Packard and various defense contractors. This methodology provided a pragmatic tool for engineering managers to prioritize maintenance efforts by assigning a tangible value to code quality. While the MI provides a high-level overview of code density, it suffers from what can be described as "semantic blindness". Metrics such as Cyclomatic Complexity and Halstead Volume analyze the control flow and token count of a file but fail to interpret the structural intent or the relationships between components. For the purposes of this paper this lack of overarching analysis will be referred to as "blind metrics". Utilising blind metrics allows for efficient and quick analysis but it opens the door to gaming the system in a sense, where developers could reach extremely maintainable scores syntactically while semantically being unmaintainable, thereby abusing the formula.

For example, A script that maintains a high MI score due to low complexity and short length, yet contain architectural flaws such as "God Objects" or tight coupling to external datasets that inhibit reuse. The formula was a product of its time: finely tuned to the teams and projects it was applied to, in the modern landscape the current formula would definitely not be accurate with how modern languages have evolved. Even though the formula could be adapted to a modern context it is clear that even a modernised version would suffer from blind metrics and is not something that should be used as only counting lines cannot yield results on the true quality of code.

An approach that would apply in a more modern context was proposed by (Xiao *et al.*, 2016), of identifying architectural debt through evolutionary analysis of a codebase (Xiao *et al.*, 2016). Where architectural debt can be used as a measure of maintainability it quantifies debt through a formalized Debt Model, using regression analysis to capture the growth rate of maintenance costs over time. The paper proposed that there are four key architectural patterns that are the main proponents of ATD, Hub, Anchor Submissive,

Anchor Dominant, and Modularity Violation. These patterns are all based on combining structural context alongside evolutionary context to create a holistic view of the codebase.

Hub: Characterised by strong, mutual coupling where the anchor file and its members have structural dependencies in both directions and strong commit coupling in at least one. This pattern often represents "spaghetti code" where a central file is overloaded with responsibilities.

Anchor Submissive: Occurs when members structurally depend on an anchor, but the anchor is historically submissive, changing whenever the members change.This typically indicates an unstable interface being forced to change by its clients.

Anchor Dominant: The reverse of submissive, where members structurally depend on the anchor, and the anchor historically dominates them, frequently propagating changes outward to its dependents.

Modularity Violation: Identified as the most common and expensive form of architectural debt across various projects. It is unique because it involves files that have no explicit structural dependencies but are frequently changed together in the project's commit history.

The four patterns were measured by performing a longitudinal study on the commit histories of large open source codebases. Using tools such as Understand and Titan to derive the relevant pattern by calculating the commit coefficient between every file.

A metric which relates to the chance of a commit on one file will require a commit on another. For example, given files A, B, and C with commit histories of {A,B} and {A,C}:

- If B or C is modified: 100% chance that A will also be modified
- If A is modified: only 50% chance that B or C will be modified

This would then be called a fileset, this would be extrapolated over the entire commit history and codebase creating many filesets. This would allow the tool to extract semantic relations between files that are not apparent structurally. The study was able to compound this effect by measuring the number of commits labeled as bug fixes against the number of feature commits, this allows simple data analytics to measure the amount of maintenance debt that each fileset would have. Using this method, high maintenance filesets can be labelled and evaluated as a quantitative metric. The core value of their work is the identification of Modularity Violations: instances where files are forced to change in tandem despite having no logical reason to do so. In the "wild west" of research software, where commit messages are often uninformative, it is not possible to rely on a human to label a change as a "bug fix." However, a substitute that can can be explored is replacing missing semantic data with the sheer physical volume of change. This necessitates shifting the focus from why a file changed to how much it changed focusing more on temporal changes, leading to the study of Code Churn.

3.2.3 Code churn's impact on maintainability

Code churn is the rate of change of lines of code in a file. A file with high churn might have an incredibly high number of additions and deletions while remaining small in terms of LOC. This is an ATD hotspot when assessing points of interest in terms of maintainability in codebases. (Faragó, Hegedűs and Ferenc, 2015) is paper which assess how code code churn affects the maintainability of codebases, by assessing the code churn of files in large codebases, achieving this by calculating the sum total of lines added and deleted on every file. Supplementing this the maintainability of the code was measured using ColumbusQM probabilistic software quality model (Bakota *et al.*, 2011). The paper was able to assess the levels of maintainability per file in proportion to the amount of code changes, drawing the conclusion that code that has high churn is code in which it is harder to maintain. This conclusion makes sense, it reaffirms that poorly architected code will need more changes which increases the maintainability burden of the code. The method in which the code churn was measured is quite useful and will be the approach taken for this study.

Unfortunately there is one key issue with this paper, the use of ColumbusQM for assessing the maintainability of the codebase. ColumbusQM is a statistical machine which takes metrics such as lines of code, styling, coupling, number of incoming invocations etc.. Aggregates them which then computes a numerical output. The metrics such as styling are something so subjective and minor that the inclusion as one of the key metrics in the evaluation underscores the whole statistical model. Taking into account all of the evaluation metrics this is a case of blind assessment where structural relations or evolutionary dependencies are not measured and only a snapshot of the codebase is measured. Compounding on this the evaluation metric for the output of the statistical model was based on developer feedback and ideas, which is unreliable and subjective.

Taking these points into account, the key takeaway from (Faragó, Hegedűs and Ferenc, 2015) is the methodology regarding code churn, rather than its conclusions on maintainability. By identifying the intersection of Modularity Violations (Structural) and High Churn (Temporal), composite metric can be created that avoids the "blindness" of ColumbusQM. This allows us to quantify architectural decay using only version control metadata, remaining entirely agnostic to both the quality of documentation and the subjective "style" of the researcher. Giving us the foundation of files to be used to create opinionated static analysis tooling.

3.3 Static Analysis

The transition from historical temporal analysis to active development requires a mechanism that can evaluate code in its current state, rather than its past. Static Analysis serves as this mechanism, offering a storied field of research focused on improving code quality by examining source code without execution. While the temporal analysis identifies the "symptoms" of architectural decay through commit history, static analysis allows us to identify the specific structural patterns that lead to high maintenance overhead.

3.3.1 Current Methods

Modern methods for static analysis include data flow analysis, syntactic pattern matching, abstract interpretation, constraint based analysis etc… (Gosain and Sharma, 2015). Using these methods present a significantly more detailed analysis of programs, where a move has been made from a token level analysis to a project level analysis, allowing approaches such as path analysis and reasoning about runtime behaviour. Yet even with this increase in analytic capabilities noise pollution in which there are too many error messages discouraging any actions being taking to remedy them, the same issue which plagued Lint (Johnson and Hill, 1978) is still an issue to this day (Dietrich, 2017). Polluting the developer experience creating friction and actively discouraging developers form using the tools as it becomes more effort to sift through and find the real bugs than fixing them normally (Johnson *et al.*, 2013). This leads to the conclusion that the research community especially due to the lack of experience in the field (Hannay *et al.*, 2009) would have more difficulties differentiating fact from fiction and discouraging them from using static analysis at all. This concretely highlights the difficulties of modern static analysis tooling in a research context. The focus on a general one size fits all tool means that it lacks the ability to be opinionated be default. The gap this presents is one where research software would have a bespoke set of curated rules reducing noise and guaranteeing improvements in code quality once executed.

This paper poses that the key issue with the analysers in the case of false positives is not the rules but how they were derived. Historically most academic research in software engineering has required significant empirical evidence to be considered by industry (Weyuker, 2011), yet the current landscape is dominated by expert analysed rules. The challenge of software maintainability with static analysis is elusive. 'Ghost Echoes Revealed' highlights this (Borg, Ezzouhri and Tornhill, 2024), critiquing the 'ghost echoes' produced by traditional expert-derived static analysis. These echoes are not objective truths, but rather technical artifacts, static rules that linger in codebases despite often failing to align with the lived experience and thoughts of human developers.

By benchmarking machine learning models against human judgment, the study reveals a fundamental difference: expert rules frequently draw 'hard lines' that humans do not actually perceive as problematic. This suggests that codebase governance based on these traditional metrics is less an exercise in objective optimization and more an adherence to institutional haunting. The precedent set here is that for codebases to remain truly maintainable, developers must move beyond the rigid, subjective 'opinions' of experts and toward empirically validated models that reflect the nuanced, non-deterministic reality of human-centric code health."(Borg, Ezzouhri and Tornhill, 2024)

3.4 Machine learning

Machine learning enables exactly this: rather than relying on expert opinion to define what "maintainable" means, we can derive rules empirically from how developers actually behave. Machine learning is a pivotal technology within the context of this paper. It

allows rigorous statistical analysis to be carried out to create objective metrics for the maintainability rules. Machine learning is based on the principle of minimising loss, creating a method of training an algorithm to optimise for the lowest loss through gradient descent allows for the code to be self improving within the problem domain. There are many approaches and models available but due to an emphasis put on human readability decision trees and random forest best fits this use case.

3.4.1 Decision Trees

Decision trees are a standout choice when it comes to an interpretable machine learning models. They consistently rank highly within classifier models while still supporting rule extraction which aids readability. Decision trees represent classifiers as hierarchical IF-THEN rules, where each root-to-leaf path corresponds to a conjunction of feature thresholds that directly maps to executable static analysis checks. A concrete example of this would be IF nesting > 3 THEN unmaintainable, in the case of nesting being over 3 then the code would be labeled unmaintainable. This simplistic example would be scaled up much larger with many more decision and leaf nodes creating a classifier based off simple decisions at each step. Naturally there are some drawbacks to this approach, most prominently is the overfitting problem. An overfitted model reflects the structure of the training data set too closely. Even though a model appears to be accurate on training data,, it may be much less accurate when applied to a current data set (Khoshgoftaar and Allen, 2001). This issue restricts their usage in current maintainability prediction methods, (Bluemke and Borsukiewicz, 2023) shows that decisions trees can prove useful in a baseline analysis of maintainability predictors. Despite limitations, DT rules provide the simple interpretability missing from random forests, enabling manual validation of learned static checks against domain knowledge. Pruning or ensemble averaging in random forests addresses overfitting to a degree while retaining path-derived rules for analysis (Quinlan, 1993).

3.4.2 Random Forest

Random forest is an approach derived from decision trees and arguably one of the most popular classifier machine learning approaches. Relying on creating a set of decision trees (called a forest) each of which vote on the outcome to choose a class for the classifier, it creates a method that leverages the law of large numbers to deal with the overfitting problem in decision trees (Breiman, 2001) in addition to increasing accuracy. The method in which overfitting can be largely ignored is that there are many trees voting, every one could potentially be overfitted yet the whole forest remains generalised to the problem as every tree would be overfitted to a different feature. Random forests average better results theoretically (Breiman, 2001) and practically in the field of maintainability research (Bluemke and Borsukiewicz, 2023) over decision trees. The key drawbacks to random forests are both the increased computational costs and the black box view when it comes to readability. Modern research (Haddouchi and Berrado, 2025) has made progress in creating methods of rule extraction for random forests as before the size of the forests rendered them as a black box approach. (Haddouchi and Berrado, 2025) focused on deriving a

rule ensemble based on the calculated weighted importance of the trees. Allowing for a dimension reductionality method to be applied to the forest. Within the study a factor of 300x reduction to the trees per class was observed while retaining a 93% accuracy compared to the full forests 95% accuracy. Leveraging this rule extraction method for random forests is pivotal to create a rule set that properly generalises across projects while retaining the core idea and performance of the model and keeping with the core ideal of readability.

3.4.3 Applications to maintainability detection

The application of machine learning to maintainability detection enables a data-driven approach to identifying code structures that hinder long-term quality and evolution. Traditional static analysis methods rely on human-curated heuristics, which often fail to generalise across diverse projects or languages. In contrast, machine learning can infer maintainability rules directly from empirical evidence, allowing patterns of poor or high maintainability to emerge statistically rather than being predefined. SotA models achieve impressive results in classification of low maintainability files. (Bertrand *et al.*, 2023) achieved an 82% F1 score with an AdaBoost classifier, while (Bluemke and Borsukiewicz, 2023) achieved 93% F1 with a random forest classifier. However, both approaches share a critical limitation, they rely on human experts to create a ground truth of annotated data. PROMISE (Boetticher, Menzies and Ostrand, 2007) and MainData (Schnappinger, Fietzke and Pretschner, 2020) were the datasets used in these studies, relying on expert annotation. This introduces subjectivity and limits scale. These datasets are also by design older and cannot reflect the current coding landscape. This limitation is the key motivation for an algorithmic approach to labelling, allowing us to create a dataset based on current codebases at scale.

A crucial step in applying these machine learning to maintainability detection is the transformation of source code into a machine-readable representation. Abstract Syntax Trees (ASTs) provide this structure, encoding the syntactic and hierarchical relationships between program elements. By traversing or analysing the AST, a wide range of numerical and categorical features can be extracted: such as nesting depth, branching density, or average method size, which serve as the input features for machine learning models (Bertrand, Favier and Andre, 2022). Various learning models can then be trained on these AST-derived metrics to classify code fragments according to maintainability characteristics.

3.5 Summary

The machine learning approach offers a compelling alternative to expert-derived static analysis rules. By using interpretable models such as Decision Trees, we can extract human-readable rules that map directly to executable code quality checks. Random Forests provide higher accuracy but require additional effort to extract interpretable rules. Prior work has demonstrated strong classification performance (82-93% F1), but these approaches rely on expert-labeled datasets that are difficult to scale and may reflect outdated coding standards.

This study takes a different approach: rather than relying on human experts to label data, we use the Hub Score an objective metric derived from version control metadata, to automatically label files as "High Risk" or "Low Risk." This automated labeling enables us to train machine learning models on a larger, more diverse dataset without the overhead of manual expert annotation. The resulting model learns which structural characteristics (nesting depth, complexity, coupling) correlate with files that developers actually change frequently, translating these patterns into static analysis rules that are empirically grounded rather than opinion-based.

4 Methodology

This study follows an empirical, data-driven pipeline to derive static analysis rules from real-world research software. The process is divided into four primary phases. First, we aggregate a dataset of C++ repositories from the Journal of Open Source Software (JOSS). Second, we employ a custom-built tool to extract evolutionary coupling metrics and calculate a Hub Score for each file, providing an objective "High Risk" or "Low Risk" label. Third, these labeled files are used to train a Decision Tree model to identify the structural characteristics of high-risk code. Finally, the logical paths within the trained model are translated into human-readable static analysis rules, bridging the gap between historical developer behavior and proactive code quality standards.

4.1 Dataset Selection and Acquisition

The primary goal of this paper is to derive objective rules to raise the level of coding standards in the research software scene. This lends itself to using research software as the dataset. The Journal of Open Source Software (JOSS) (*Journal of Open Source Software*, no date) was selected as the primary source due to its peer-review requirement, which ensures a baseline of documentation and code quality.

4.1.1 Data collection

A custom utility tool in rust was created to aggregate metadata from JOSS. This tool interfaced with Github API to gather repository-level metrics(contributor count, collaborator count). The resulting data was analysed by a python pipeline with pandas (team, 2020) to determine the final dataset. To ensure sufficient architectural complexity, projects were filtered based on the following criteria:

- Collaborator Threshold($>= 4$): Projects with fewer than four collaborators often reflect individual coding styles rather than standardized maintainability practices. A minimum of four contributors ensures a level of communication overhead that requires formal architectural patterns. As seen in Figure 1 the majority of repositories remain after this filtering, while low-developer outliers are removed.

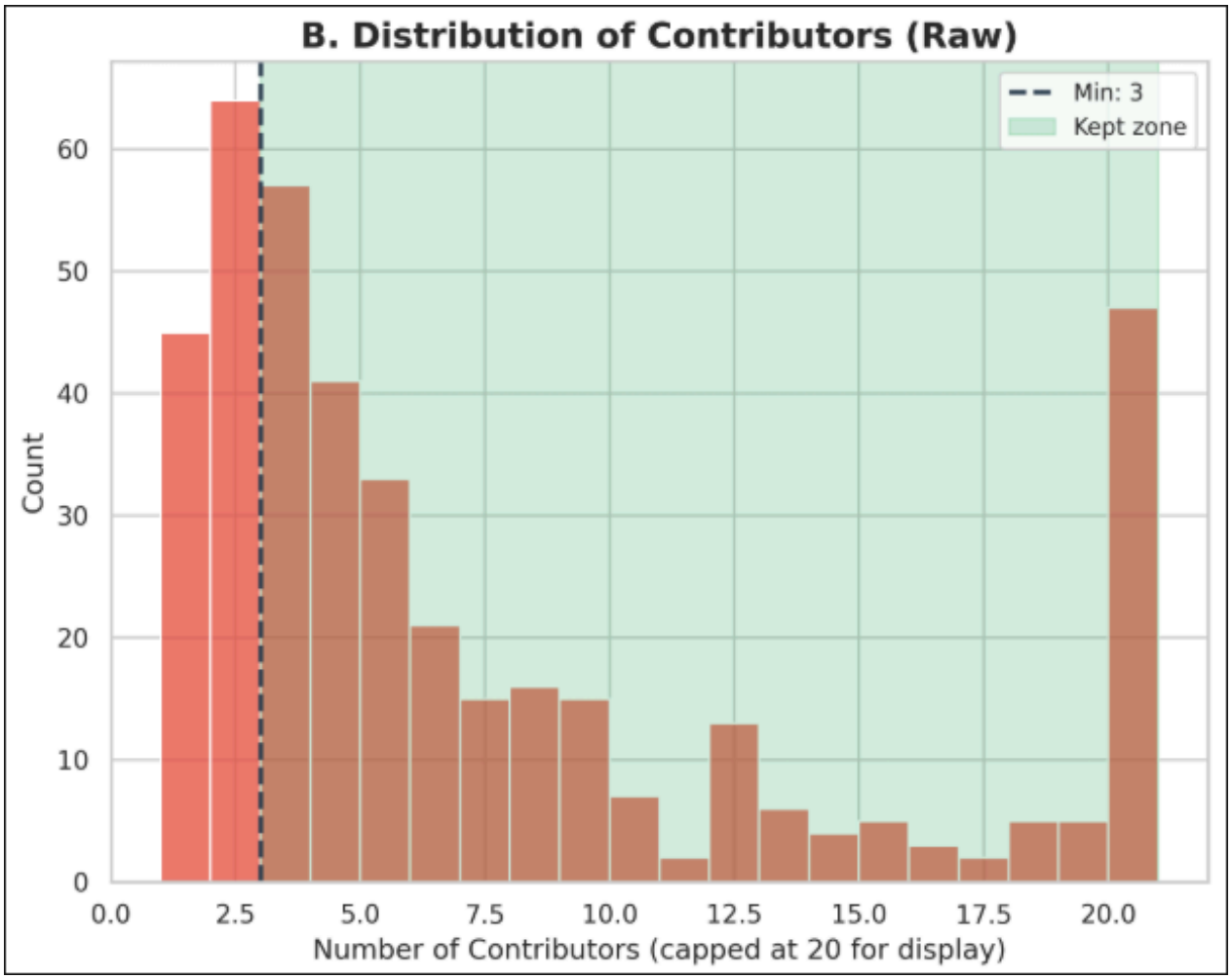

Figure 1: Pre filter Distribution of Collaborators

- Commit Volume (100–5,000): To ensure the dataset captures meaningful maintenance behavior, a "maturity floor" of 100 commits was established. Conversely, a "computational ceiling" of 5,000 commits was enforced to maintain feasibility on local hardware. As seen in Figure 2, these thresholds effectively remove the "short-tail" of embryonic projects and the "long-tail" of infrastructure-scale outliers. This dual-sided filtering retains the vast majority of the JOSS population while ensuring a consistent and processable data scale.

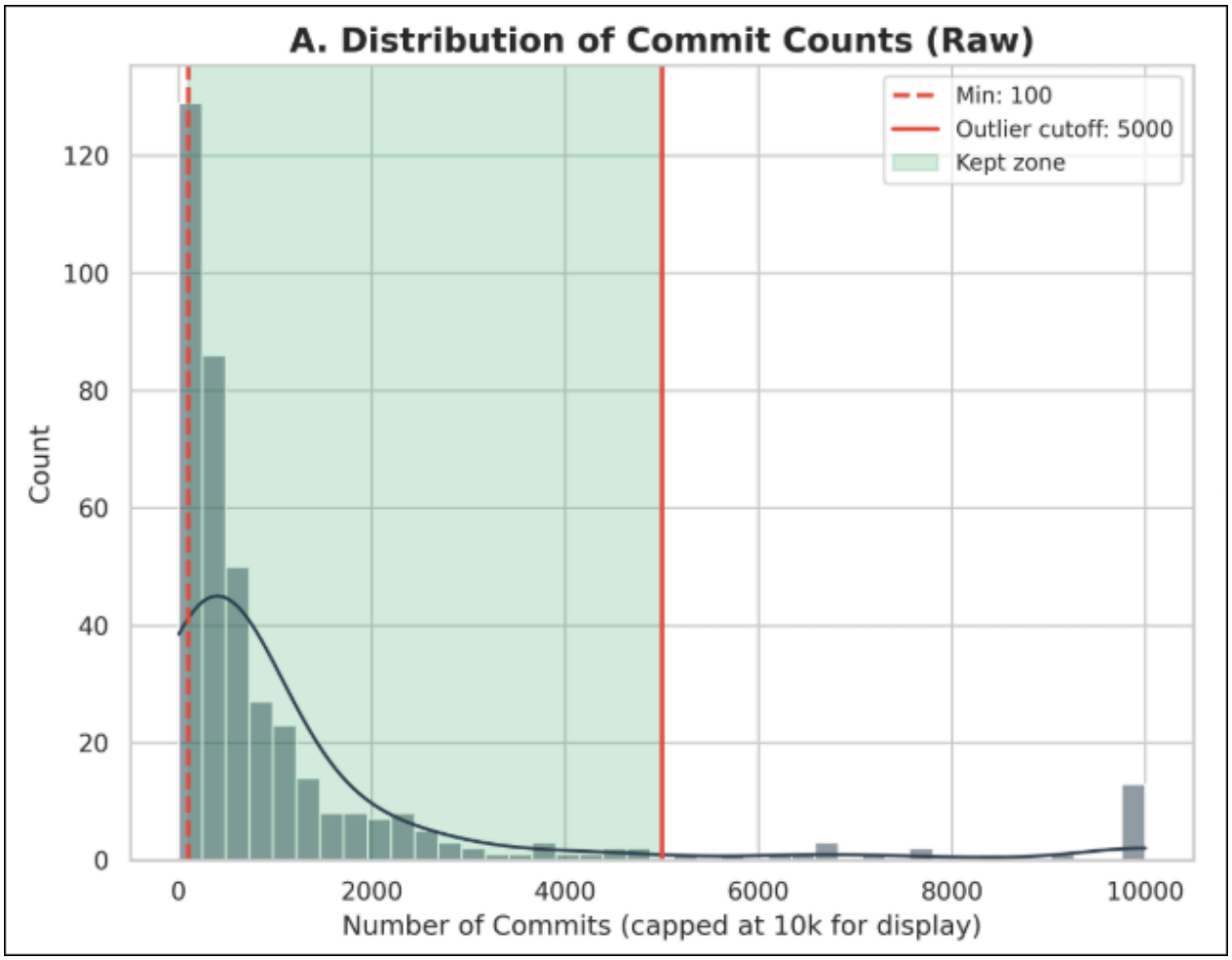

Figure 2: Pre filter Distribution of Commits

- Target Language: C++ was selected as the target language due to its dominance in high-performance research computing and the specific maintainability risks associated with its manual memory management and low-level abstractions. Restricting the study to a single language ensures methodological consistency, as code metrics are often non-comparable across different programming paradigms and syntax structures.

This filtering took the original 406 repos down to 213 which is a significant reduction in dataset size, those projects primarily consisted of individual small scale work that lacks the depth and collaboration overhead required to test maintainability work.

4.1.2 Dataset First Pass Analysis

To categorize the dataset,automated labeling heuristics were utilised rather than traditional expert manual review. While expert opinion is often the standard for establishing "ground truth," it is difficult to scale across thousands of research repositories. Instead, the tool analyzed the historical behavior of each file to assign a Hub Score: a composite metric that measures a file's "dependency" within the project. This score is calculated based on three key factors: the number of coupled files, the average coupling ratio, and the file's overall

code churn. These metrics were gathered by running a custom analysis tool over the entire commit history of each repository. The tool constructs a Co-change Graph where, Nodes represent individual files and Edges represent shared commits between files. The graph allows us to evaluate the coupling of each file by comparing the number of times it was modified alongside other files versus the number of times it was modified in isolation.

To map the evolving relationships between source files a custom rust tool was created. Due to the scale of the relationships between files that needed to be tracked an in memory solution was not feasible, therefore Neo4j (Technology, 2015) was an ideal solution. As a graph database, Neo4j is uniquely suited for managing the complex, non-linear relationships inherent in software evolution. The tool performs a sequential traversal of the repository's entire commit history from the initial root, to the current head. At each commit tracking the codependency between each file as they were committed over time. This created a robust database of the temporal dependencies of the repository's architectures. A key technical challenge arose over large commits. In research software researchers wouldn't be aware of commit hygiene (Hannay *et al.*, 2009) leading to large scale "bulk" commits. If a dependency such as a dataset were uploaded of 1,000 files, that could generate 1,000,000 relationships to track. To maintain performance and ensure data relevance, a threshold was established to skip any commit containing more than 100 files. This optimization focuses the database on intentional, developer-driven architectural changes rather than bulk file operations.

The output of the analysis pipeline was a Neo4j graph database, which served as the computational foundation for calculating file-level maintainability risks. By leveraging graph-traversal queries, the dataset was categorized using a composite heuristic "Hub Score."

4.2 Dataset Labeling

Drawing on three complementary insights from prior work, this paper proposes a composite Hub Score that synthesizes temporal coupling and change frequency into a single normalized metric. (Xiao *et al.*, 2016) demonstrated that commit coefficients between files serve as reliable indicators of architectural debt. Files that frequently change together despite lacking structural dependencies represent "Modularity Violations" that accumulate ATD. Building on this, (Faragó, Hegedűs and Ferenc, 2015) showed that files exhibiting high code churn (frequent additions/deletions) correlated directly with reduced maintainability. Finally, (Cai *et al.*, 2025) empirically validated that coupling density measured by the number of partner files directly increases maintenance overhead across 1,200 Google repositories.

As noted by (Xiao *et al.*, 2016), focusing primarily on modularity violations ignores other potential risk factors. However, this approach is justified by the unique scale of our study. Unlike traditional, manually-curated datasets such as PROMISE (Boetticher, Menzies and Ostrand, 2007) or MainData (Schnappinger, Fietzke and Pretschner, 2020), our automated "broad-spectrum" filtering allows us to identify the most critical offenders

across a significantly larger volume of data. By combining these repositories, a dataset was created that is both objective and massive in scope, providing a high-quality surface area for supervised learning using "High Risk" and "Low Risk" labels.== Feature Selection. To quickly calculate a relative score of each file this formula was used to calculate a value called a Hub Score which represents the temporal coupling and change rate of every file in compared to others.

4.2.1 Hub Score

$$\text{HubScore} = |(C) \times \left(\frac{P}{F_t}\right) \times \left(\frac{W_f}{W_t}\right) \tag{2}$$

Variable Definitions

- $|(C)$ (**avg_coupling**): The mean strength or frequency of dependencies between this file and its partners.
- P (**partner_count**): The number of unique files that this specific file is coupled with.
- F_t (**total_files**): The total number of files in the repository or local subset.
- W_f (**file_churn**): The number of changes or commits associated with this specific file.
- W_t (**total_churn**): The total sum of changes across the entire repository.

This is a bespoke formula normalised to allow accurate comparison across repositories. The core justification to use this formula is derived from three core principles:

1. A logic file should not have to change often. Supported by the Open Closed principle and (Faragó, Hegedűs and Ferenc, 2015), which demonstrated that high frequency and large size of changes correlate to an increased maintenance overhead.
2. A logic file should not be highly coupled to many partner files. (Cai *et al.*, 2025) empirically validated that coupling density directly increases maintenance overhead across 1,200 Google repositories.
3. Files that change together despite lacking structural dependencies represent hidden architectural debt. (Xiao *et al.*, 2016) identified these "Modularity Violations" as the most common and expensive form of architectural debt, where files co-change without explicit dependencies.

This three-principle justification is not merely theoretical; it mirrors the approach of prior work that uses evolution history as ground truth for supervised learning. demonstrated that co-change information can produce higher-quality labels for code smell detection than expert annotation alone, validating the use of temporal coupling as an objective labeling mechanism for machine learning. The formula was further substantiated where when benchmarking against sonarqube high risk files identified by sonarqube were also identified by this formula, while not all high risk files were identified, each of the high risk files identified were associated with an above average maintenance burden which evidences the ability of it.

4.2.2 Contrastive sampling

To prepare the data for a binary Decision Tree classifier, the continuous Hub Score was split into two categorical classes: High Risk and Low Risk. Rather than utilizing a static

numerical threshold, which may vary significantly across project scales, a Global Rank-Ordering strategy was applied:High Risk Class ($Y = 1$): Defined as the top 2,000 files ($N_{\{\top\}}$) across the entire processed dataset. These files represent the extreme outliers in terms of coupling and change frequency. Low Risk Class ($Y = 0$): To ensure a clear decision boundary for the classifier, Contrastive Under-sampling was used. An equivalent $N = 2,000$ files were sampled from the bottom quartile of the Hub Score distribution.By using the "middle-ground" files (those in the 25th percentile), we maximize the variance between classes while using files that have some importance temporally, allowing the Decision Tree to more effectively identify the structural signatures of high-maintenance code.

4.2.3 Addressing Data Leakage

A critical consideration in this methodology is the separation of Labeling Criteria and Training Features. The Labels are derived from Temporal Metadata (commit history, co-change frequencies, and historical churn). The Features (inputs for the Decision Tree) are derived strictly from Static Source Code Analysis (e.g., McCabe Complexity, LoC, Nesting Depth). This separation ensures that the model is learning to predict future maintenance risk based on the current state of the code, rather than simply "re-discovering" the components of the Hub Score formula.

4.2.4 Dataset Summary

The final balanced dataset consists of 4,000 observations. This 50/50 class distribution prevents the classifier from developing a majority-class bias, ensuring that the resulting decision rules are equally sensitive to the characteristics of "Hub" files.

4.3 Feature Engineering

To prepare the high- and low-risk datasets for machine learning, the raw source code files had to be converted into tabular data compatible with decision trees and random forests. Feature extraction was performed using the rust-code-analysis (RCA) crate (Ardito *et al.*, 2020). RCA is used on the base source files to statically derive numerical metrics on the files, the metrics can be seen in Table 1

Metric	Description
CC	McCabe's cyclomatic complexity: it calculates the code complexity examining the control flow of a program. It is measured as the number of linearly independent paths through a piece of code.
SLOC	Source lines of code: it returns the total number of lines in a source file.
PLOC	Physical lines of code: it returns the total number of instruction and comment lines in a source file.
LLOC	Logical lines of code: it returns the total number of logical lines (statements) in a source file.
CLOC	Comment lines of code: it returns the total number of comment lines in a file.
BLANK	Blank lines: it counts the number of blank lines in a source file.
Halstead	The Halstead suite, a set of seven statically computed metrics, all based on the number of distinct operators (n1) and operands (n2) and the total number of operators (N1) and operands (N2). The suite provides a series of information, such as the effort required to maintain the analyzed code, the size in bits to store the program, the difficulty to understand the code, an estimate of the number of bugs present in the codebase, and an estimate of the time needed to implement the software.
MI	Maintainability index: a suite to measure software's maintainability, calculated both on source files and functions.
NOM	Number of Methods: it returns the number of methods in a source file.
NARGS	Number of arguments: it counts the number of arguments of each method in a source file.
NEXITS	Number of exits: it counts the number of possible exit points of each method in a source file.

Table 1: Extracted code metrics

Prior to model training the extracted featured underwent additional preprocessing to enhance performance. Columns containing NaN values were removed, as certain language-specific metrics were non-applicable to the C++ source files within the dataset. The Maintainability Index (MI) column was dropped to remove potential bias from previous maintainability prediction methods. Low variance (<0.01) and highly correlated features ($|r| < 0.9$) were dropped to reduce the search space when selecting variables. Simple ratios between relevant features (Coleman *et al.*, 1994) were computed such as operator operand ratio to capture relative relationships between measurements. Lastly due to the nature of tree based machine learning models further metrics were calculated using sklearn's (Pedregosa *et al.*, 2011) polynomial feature generation on key numerical metrics to allow non linear metrics to be an option for the tree model feature selection.

4.4 Model Implementation and Training

Following the refinement of the feature set, the processed tabular data was utilized to train and validate two supervised learning models: a Decision Tree and a Random Forest. These models were selected for their inherent interpretability, allowing the extracted maintainability metrics to be mapped to clear, logical decision boundaries.

To ensure a robust evaluation of both models, a standardised training approach was used. Sklearns's (Pedregosa *et al.*, 2011) DecisionTreeClassifier and RandomForestClassifier were used as the base models. GridSearchCV was used to perform hyperparameter fine tuning on the models where F1 score was used as the primary optimisation target ensuring a balanced performance metric between high and low risk files. A 5-fold stratified cross-validation was used over a traditional train-test split to ensure that each fold remained representative of the overall class distribution, mitigating the risk of sampling bias that can occur with a single split Coupled with this to aid in the recall of the model to lower the chance of a false negative, a weighted cost function was applied, penalizing the misclassification of high-risk files more heavily than low-risk files.

The optimal configurations for decision trees and random forests are detailed in the results section.

4.5 Rule Derivation Process

The selection of tree-based models was driven by the requirement for rule extraction. Once the models reached optimal performance, two distinct methodologies were used to translate the mathematical weights into human-readable coding standards:

- Decision trees: Decision trees provide an inherent "IF-THEN" logical structure. To ensure interpretability, the models were constrained by maximum depth parameters during training. The trained model was exported as a visual representation using Scikit-learn's tree visualization tools (Pedregosa *et al.*, 2011), allowing for a clear view of the decision paths.
- Random Forests: Due to the complex ensemble nature of random forests only a derivation of the rules can be extracted (Haddouchi and Berrado, 2025). Using the tool te2rules (Lal, Chen and Mithal, 2024) an approximation of the random forests rules can be generated to a high fidelity providing a human-readable summary of the ensemble's collective logic.

The extraction on human readable rules is a strong step towards practical application. However they cannot be trusted without empirical testing as without it they are worse than expert rules.

4.6 Evaluation Framework

To determine the validity of the ML-derived rules testing against the current Sota in static analysis is required. This method will test

6. Outputs

- Confusion matrices (per-model CV)
- Model comparison bar chart
- Decision tree visualization
- Per-repo performance metrics (CSV)

5 Future Work

- This should be a black box model that can work in AI loops where a false positive is less impactful
- Use larger dataset samples and apply it to more domains
- Use other models that are not human explainable
- Look at deleted files
- identify high risk files and pull in the related files to help build patterns

Bibliography

Ardito, L. *et al.* (2020) "rust-code-analysis: A Rust library to analyze and extract maintainability information from source codes," *SoftwareX*, 12, p. 100635. Available at: https://doi.org/10.1016/j.softx.2020.100635.

Bakota, T. *et al.* (2011) "A probabilistic software quality model," in, pp. 243–252. Available at: https://doi.org/10.1109/ICSM.2011.6080791.

Bertrand, S. *et al.* (2023) "Replication and Extension of Schnappinger's Study on Human-level Ordinal Maintainability Prediction Based on Static Code Metrics," *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pp. 241–246. Available at: https://doi.org/10.1145/3593434.3593488.

Bertrand, S., Favier, P.-A. and Andre, j.-m. (2022) "Building an Operable Graph Representation of a Java Program as a Basis for Automatic Software Maintainability Analysis," in, pp. 243–248. Available at: https://doi.org/10.1145/3530019.3534081.

Bluemke, I. and Borsukiewicz, P. (2023) "Experiments on software error prediction using Decision Tree and Random Forest algorithms", pp. 865–869. Available at: https://doi.org/10.15439/2023F363.

Boetticher, G., Menzies, T. and Ostrand, T. (2007) "\textbackslash{PROMISE\textbackslash} Repository of empirical software engineering data."

Borg, M., Ezzouhri, M. and Tornhill, A. (2024) *Ghost Echoes Revealed: Benchmarking Maintainability Metrics and Machine Learning Predictions Against Human Assessments*. arXiv. Available at: https://doi.org/This was measured by preforming a pseudo longitudinal study on large open source codebases, using tools such as Understand and

Titan to calculate commit coefficient between every file. A metric which relates to the chance of a commit on one file will require a commit on another. An example would be given files A, B and C, with a commit history of A,B and A,C the commit coefficients in relation to A are there is a 100% chance if B or C is modified A will be also, but if A is modified there is only a 50% chance that B or C will be modified. This would then be called a fileset, this would be extrapolated over the entire commit history and codebase creating many filesets. This would allow the tool to extract semantic relations between files that are not apparent structurally. The study was able to compound this effect by measuring the number of commits labeled as bug fixes against the number of feature commits, this allows simple data analytics to measure the amount of maintenance debt that each fileset would have. Using this method, high maintenance filsets can be labelled and evaluated as a quantitative metric. This is a strong contender for a evaluation metric but there is a caveat, that is it is reliant on good commit messages and issue tracking which is not commonly seen in research code. Meaning that a new approach that is agnostic to the quality of the codebase is required to. === Code churn's impact on maintainability Code churn is the rate of change of lines of code in a file. A file with high churn might have an incredibly high number of additions and deletions while remaining small. This is an ATD hotspot when assessing points of interest in terms of maintainability in codebases. @farago_cumulative_2015 is paper which assess how code code churn affects the maintainability of codebases, by assessing the code churn of files in large codebases, achieving this by calculating the sum total of lines added and deleted on every file. Supplementing this the maintainability of the code was measured using ColumbusQM probabilistic software quality model @noauthor_pdf_nodate. The paper was able to assess the levels of maintainability per file in proportion to the amount of code changes, drawing the conclusion that code that has high churn is code in which it is harder to maintain. This conclusion makes sense, it reaffirms that poorly architect ed code will need more changes which increases the maintainability burden of the code. The method in which the code churn was measured is quite useful and will be the approach taken for this study. Unfortunately there is one key issue with this paper, the use of ColumbusQM for assessing the maintainability of the codebase. ColumbusQM is a statistical machine which takes metrics such as lines of code, styling, coupling, number of incoming invocations etc.. Aggregates them which then computes a numerical output. The metrics such as styling are something so subjective and minor that the inclusion as one of the key metrics in the evaluation underscores the whole statistical model. Taking into account all of the evaluation metrics this is a case of blind assessment where structural relations or evolutionary dependencies are not measured and only a snapshot of the codebase is measured. Compounding on this the evaluation metric for the output of the statistical model was based on developer feedback and ideas, which is unreliable and subjective. Taking these points into account, the key takeaway from @farago_cumulative_2015 is the work on code churn particularly the methodology but the conclusions in regards to maintainability will only be used as an idea within this study..

Breiman, L. (2001) “Random Forests,” *Machine Learning*, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Cai, Y. *et al.* (2025) “Understanding Architectural Complexity, Maintenance Burden, and Developer Sentiment -A Large-Scale Study,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2176–2187. Available at: https://doi.org/10.1109/ICSE55347.2025.00168.

Coleman, D. *et al.* (1994) “Using Metrics to Evaluate Software System Maintainability,” *Computer*, 27, pp. 44–49. Available at: https://doi.org/10.1109/2.303623.

Dietrich, E. (2017) *How Has Static Code Analysis Changed Through the Years?*. Available at: https://blog.ndepend.com/static-code-analysis-changed-years/ (Accessed: February 22, 2026).

Faragó, C., Hegedűs, P. and Ferenc, R. (2015) “Cumulative code churn: Impact on maintainability,” in *2015 IEEE 15th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pp. 141–150. Available at: https://doi.org/10.1109/SCAM.2015.7335410.

Gosain, A. and Sharma, G. (2015) “Static Analysis: A Survey of Techniques and Tools,” in D. Mandal et al. (eds.) *Intelligent Computing and Applications*. New Delhi: Springer India, pp. 581–591. Available at: https://doi.org/10.1007/978-81-322-2268-2_59.

Haddouchi, M. and Berrado, A. (2025) “Forest-ORE: Mining an optimal rule ensemble to interpret random forest models,” *Engineering Applications of Artificial Intelligence*, 143, p. 109997. Available at: https://doi.org/10.1016/j.engappai.2024.109997.

Hannay, J.E. *et al.* (2009) “How do scientists develop and use scientific software?,” in *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pp. 1–8. Available at: https://doi.org/10.1109/SECSE.2009.5069155.

Howison, J. and Herbsleb, J.D. (2014) “The sustainability of scientific software:,” *2014* [Preprint]. Available at: https://doi.org/https://james.howison.name/pubs/HowisonHerbsleb-Sustainability.pdf.

ISO 25010 (no date). Available at: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (Accessed: February 2, 2026).

Jiménez, R.C. *et al.* (2017) *Four simple recommendations to encourage best practices in research software*. F1000Research. Available at: https://doi.org/10.12688/f1000research.11407.1.

Johnson, B. *et al.* (2013) “Why don't software developers use static analysis tools to find bugs?,” in *2013 35th International Conference on Software Engineering (ICSE)*, pp. 672–681. Available at: https://doi.org/10.1109/ICSE.2013.6606613.

Johnson, S. and Hill, M. (1978) “Lint, a C Program Checker,” in. Available at: https://www.semanticscholar.org/paper/Lint%2C-a-C-Program-Checker-Johnson-Hill/74617cffa3c6438d04aa99bef1cca415de47d0d3 (Accessed: February 8, 2026).

Journal of Open Source Software (no date). Available at: https://joss.theoj.org/ (Accessed: April 2, 2026).

Khoshgoftaar, T.M. and Allen, E.B. (2001) "Controlling Overfitting in Classification-Tree Models of Software Quality," *Empirical Software Engineering*, 6(1), pp. 59–79. Available at: https://doi.org/10.1023/A:1009803004576.

Lal, G.R., Chen, X. and Mithal, V. (2024) *TE2Rules: Explaining Tree Ensembles using Rules*. Available at: https://arxiv.org/abs/2206.14359.

Marwick, B. (2017) "Computational Reproducibility in Archaeological Research: Basic Principles and a Case Study of Their Implementation," *Journal of Archaeological Method and Theory*, 24(2), pp. 424–450. Available at: https://doi.org/10.1007/s10816-015-9272-9.

Oman, P. and Hagemeister, J. (1992) "Metrics for assessing a software system's maintainability," in *Proceedings Conference on Software Maintenance 1992*, pp. 337–344. Available at: https://doi.org/10.1109/ICSM.1992.242525.

Parnas, D.L. (1972) "On the criteria to be used in decomposing systems into modules," *Commun. ACM*, 15(12), pp. 1053–1058. Available at: https://doi.org/10.1145/361598.361623.

Pedregosa, F. *et al.* (2011) "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 12, pp. 2825–2830.

Quinlan, J.R. (1993) *C4.5: programs for machine learning*. San Mateo, Calif: Morgan Kaufmann Publishers (The Morgan Kaufmann series in machine learning).

Schnappinger, M., Fietzke, A. and Pretschner, A. (2020) "Defining a Software Maintainability Dataset: Collecting, Aggregating and Analysing Expert Evaluations of Software Maintainability," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 278–289. Available at: https://doi.org/10.1109/ICSME46990.2020.00035.

team, T. pandas development (2020) *pandas-dev/pandas: Pandas*. Zenodo. Available at: https://doi.org/10.5281/zenodo.3509134.

Technology, I. (2015) "Neo4j, the World's Leading Graph Database," *Neo4j Graph Database*, p. .

Trisovic, A. *et al.* (2022) "A large-scale study on research code quality and execution," *Scientific Data*, 9, p. 60. Available at: https://doi.org/10.1038/s41597-022-01143-6.

Weyuker, E.J. (2011) "Empirical Software Engineering Research - The Good, The Bad, The Ugly," in *2011 International Symposium on Empirical Software Engineering and Measurement*, pp. 1–9. Available at: https://doi.org/10.1109/ESEM.2011.66.

Wilson, G. (2006) "Software Carpentry: Getting Scientists to Write Better Code by Making Them More Productive," *Computing in Science & Engineering*, 8(6), pp. 66–69. Available at: https://doi.org/10.1109/MCSE.2006.122.

Wilson, G. *et al.* (2017) "Good enough practices in scientific computing," *PLOS Computational Biology*, 13(6), pp. 1–20. Available at: https://doi.org/10.1371/journal.pcbi.1005510.

Xiao, L. *et al.* (2016) "Identifying and quantifying architectural debt," in *Proceedings of the 38th International Conference on Software Engineering*. Austin Texas: ACM, pp. 488–498. Available at: https://doi.org/10.1145/2884781.2884822.